Honeypot with Attack Visualization Dashboard

Complete Project Documentation & Demonstration Manual

B.Tech Project — I · VII Semester

Department of Computer Science, Information Technology & Cyber Security

Govt. Engineering College, Ajmer · Session 2026–2027

Technology: Python · Flask · SQLite · Chart.js · Paramiko · Docker

Contents

The following sections cover the complete design, implementation, demonstration procedure and evaluation of the project.

1. Introduction & Project Overview
2. Objectives & Scope
3. System Architecture
4. How the Project Works (Event Lifecycle)
5. Module-by-Module Breakdown
6. Threat Scoring Engine
7. Database Schema
8. Technology Stack & Rationale
9. Installation & Setup
10. Starting the Demonstration (Step-by-Step)
11. Simulating Attacks Locally
12. Simulating Attacks From Another Laptop
13. The Live Demo Script (Scenario-by-Scenario)
14. Dashboard Tour
15. Security Report Generation
16. Testing & Evaluation
17. Safety, Ethics & Legal
18. Troubleshooting & FAQ
19. Limitations & Future Scope
20. References

Appendix A — Project File Structure

Appendix B — Command Quick Reference

1. Introduction & Project Overview

A **honeypot** is a controlled decoy system deliberately exposed to the network to attract, record and study unauthorized access attempts. Because it performs no real business function, **any interaction with it is suspicious by definition** and becomes useful security intelligence.

HoneyScope is a low-interaction honeypot laboratory built for this project. It exposes several safe decoy services (SSH, HTTP, FTP, Telnet and 17 silent TCP probe sensors) inside an isolated environment. Every interaction is converted into a structured event, scored by a rule-based threat engine, stored in SQLite and visualised on a live web dashboard. A one-click report generator produces a self-contained HTML security report.

The project runs entirely on a normal laptop at **zero cost**, requires no administrator/root rights with the default configuration, works offline, and enforces strict ethical guardrails. Nothing is pre-loaded or fabricated: the dashboard starts empty and only ever displays genuinely captured network traffic.

---

2. Objectives & Scope

The objectives, mapped directly to the project synopsis, are:

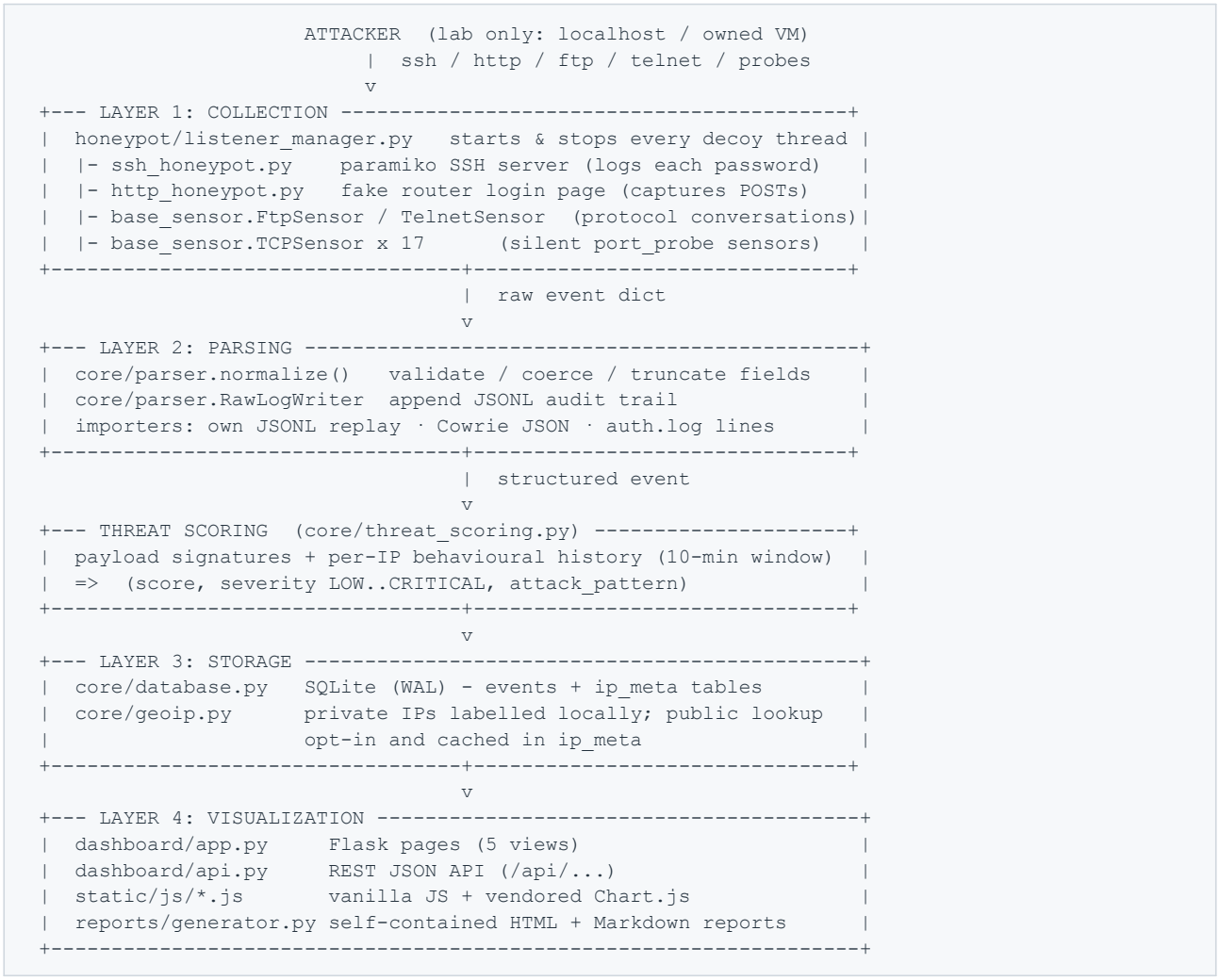
#	Objective	How it is achieved
1	Deploy a safe honeypot service in a local VM / Docker / authorized isolated machine	Five decoy service types run as threads; loopback-only by default; Docker image provided.
2	Capture attack metadata (IP, port, timestamp, protocol, credentials, payload samples)	Every decoy emits structured events; payloads truncated to 4 KB; raw JSONL audit trail kept.
3	Classify events by severity and attack pattern	Rule engine assigns LOW/MEDIUM/HIGH/CRITICAL plus brute_force, credential_stuffing, port_scan, sql_injection, xss, path_traversal, webshell_probing, scanner_tool.
4	Build a dashboard with charts, timeline, IP table, top credentials, geolocation	Flask + Chart.js dashboard: KPI cards, timeline, severity/service charts, top attackers, credential analytics, IP intel with privacy-first geolocation.
5	Generate a security report with trends and recommendations	Self-contained HTML report (charts embedded) + Markdown summary + CSV/JSON dataset export.

Scope: the system is a defensive observation and analysis instrument. It is used only in controlled labs, local networks, test VMs or authorized servers. It focuses on logging, analysis and defensive reporting — never exploitation. It can later be extended with threat-intelligence feeds, automatic blocking and alerting.

---

3. System Architecture

The project follows the four-layer design of the synopsis — collection, parsing, storage and visualization — with a threat-scoring stage between parsing and storage:



All components communicate through a single funnel — core/collector.py (the “Collection Pipeline”) — so the behaviour of live capture and historical log replay is identical.

4. How the Project Works (Event Lifecycle)

Every captured interaction passes through the same five stages. The SSH brute-force scenario is used as a running example:

Stage	What happens	Example
1. Capture	A decoy listener accepts a connection / request and emits a raw event dict	Attacker connects to :2222; paramiko performs a real SSH handshake; check_auth_password() fires with (root, toor).
2. Normalise	core/parser.normalize() validates types, strips/truncates fields, fills defaults	{service:'ssh', event_type:'login_attempt', username:'root', password:'toor', ...}
3. Score	threat_scoring.evaluate() reads the source IP's last 10 minutes and computes score + severity + pattern	The 5th consecutive failure earns +8 (brute_force); ≥6 distinct usernames earns +14 (stuffing).
4. Store	collector appends the raw JSONL line and inserts the row; ip_meta is updated	Row inserted into events; 127.0.0.1 counter and max-severity updated in ip_meta.
5. Visualise	The dashboard's 5-second poll picks up the new row via the REST API	The event appears on Overview's live feed and in the Events table.

HIGH/CRITICAL events additionally trigger a console alert (run.py: alert_console) so the operator sees escalations instantly, even without the browser.

Scoring window note: the behavioural history window is anchored to the incoming event's own timestamp, not to wall-clock time. This means imported/replayed historical logs are scored against their own era — the same result as if they had been captured live (unit-tested).

---

5. Module-by-Module Breakdown

5.1 Honeypot listeners (honeypot/)

File	Responsibility
ssh_honeypot.py	Paramiko SSH server on :2222. Generates an ECDSA host key, fakes an OpenSSH banner, records every password-authentication attempt. Optionally (accept_any_credentials=true) accepts logins and serves a sandboxed fake shell whose replies are hard-coded strings — commands are logged but NEVER executed.
http_honeypot.py	ThreadingHTTPServer on :8081 serving a realistic router-admin login page. Captures every request (method, path, query, headers, body); POSTs to /login become login_attempt events with credentials.
base_sensor.py	Generic TCP sensor (silent port_probe capture), plus FTP and Telnet decoys that hold full protocol conversations (USER/PASS, login prompts).
listener_manager.py	Builds, starts, stops and reports status for every decoy. Busy ports are reported as stopped instead of crashing startup.

5.2 Core (core/)

File	Responsibility
config.py	Defaults + config.json overrides + environment variables; directory setup.
parser.py	Event normalisation; JSONL audit writer; importers for Cowrie JSON and Linux auth.log.
threat_scoring.py	Scoring weights, payload signatures, behavioural rules, severity thresholds.
database.py	SQLite schema, indexed queries, pagination, per-IP aggregates, timeline/stats aggregation.
geoip.py	Classifies private/loopback/reserved IPs locally; opt-in public lookup with caching.
collector.py	The pipeline funnel: normalize → score → enrich → store, with alert hooks.

5.3 Dashboard & reports

File	Responsibility
dashboard/app.py	Flask application factory and the five page routes.
dashboard/api.py	REST endpoints: /api/stats, /api/events, /api/top/*, /api/intel, /api/export/*, /api/report/*, /api/reset, /api/status.
dashboard/templates/	Jinja2 pages: index, events, intel, credentials, report.
dashboard/static/	style.css (dark SOC theme) and dashboard.js (Chart.js rendering, polling, filters, modal). Chart.js is vendored for offline use.
reports/generator.py	Builds the self-contained HTML report and Markdown summary.

5.4 Scripts & tooling

File	Responsibility
run.py	Main entry point: decoys + dashboard together, or any single mode; reset-db and log-import helpers.
scripts/live_demo.py	Narrated, interactive real-attack demonstration for teachers.
scripts/simulate_attacks.py	Controlled attack simulator (real traffic) with per-scenario source addresses.
scripts/seed_demo_data.py	OPTIONAL synthetic dataset for offline preview — clearly labelled, never used in a live demo.
scripts/generate_pdf.py	Builds this PDF document.
tests/	34 unit + integration tests (unittest).

---

6. Threat Scoring Engine

Each event receives a numeric score; severity is derived from fixed thresholds. Scores combine a base event weight, payload-signature bonuses and per-IP behavioural bonuses.

6.1 Base weights

event_type	Weight	event_type	Weight
login_success	6.0	command	4.0
login_attempt	3.0	http_request	1.0
port_probe	1.0	connection / banner_grab	0.5

6.2 Payload / signature bonuses

Signature	Bonus	Detection example
SQL injection	+7	"1' OR '1'='1 UNION SELECT ..." · "--" comments · sleep()
XSS	+6	<script> · javascript: · onerror= · alert(
Path traversal	+6	../ · %2e%2e · /etc/passwd · .env
Webshell probing	+5	/cmd.php · /wso.php · phpmyadmin · wp-login.php
Scanner user-agent	+4	sqlmap · nikto · nmap · gobuster · hydra

6.3 Behavioural rules (per source IP, 10-minute window)

Rule	Threshold	Bonus	Pattern label
Brute force	≥ 5 failed logins	+8	brute_force
Credential stuffing	≥ 6 distinct usernames	+14	credential_stuffing
Port scan	≥ 5 distinct destination ports	+8	port_scan
Login velocity	each prior failure (cap 10)	+1.2 each	—

6.4 Severity thresholds

```
score < 8    -> LOW        8  <= score < 18  -> MEDIUM
18 <= score < 32 -> HIGH       score >= 32     -> CRITICAL
```

The most specific label wins when several apply: web payload patterns are reported ahead of stuffing > brute force > scan. Example escalation: a single failed login scores 3 (LOW); the 5th consecutive failure scores ≈ 15.8 (MEDIUM, brute_force); a stuffing burst reaches ≈ 32+ (CRITICAL, credential_stuffing).

---

7. Database Schema

SQLite is used with WAL journal mode and a single connection protected by a lock. Two tables store everything:

7.1 events — one row per captured interaction

Column	Type	Purpose
id	INTEGER PK	Autoincrement identifier.
ts / epoch	TEXT / REAL	UTC ISO-8601 timestamp + unix time.
src_ip, src_port, dst_port	TEXT/INT	Connection tuple.
protocol	TEXT	tcp.
service	TEXT	ssh / http / ftp / telnet / sensor.
event_type	TEXT	login_attempt / login_success / command / http_request / port_probe / banner_grab / connection.
username, password	TEXT	Captured credentials (nullable).
payload	TEXT	Request/command/probe content, truncated to 4096 chars.
user_agent	TEXT	HTTP client signature.
severity, score, attack_pattern	—	Scoring output.
geo_country, geo_city, geo_asn, geo_lat, geo_lon	—	Geolocation enrichment.

7.2 ip_meta — one row per source IP

Stores first_seen, last_seen, total_events, max_severity, an is_private flag and cached country/city/ASN/lat/lon. Indexes exist on epoch, src_ip, severity and service for fast dashboard queries.

---

8. Technology Stack & Rationale

Technology	Used for	Why it was chosen
Python 3.10+	Entire backend, decoys, scoring, tooling	Readable, batteries-included (socket, http.server, sqlite3, threading, unittest); ideal for a teaching project; single language end-to-end.

Technology	Used for	Why it was chosen
Flask	Web dashboard & REST API	Lightweight and explicit — a small API + templates + static files with full control over polling endpoints; simpler to audit than Django; a single dependency-light process.
SQLite	Event storage	Zero-administration, file-based, ACID, SQL analytics, WAL concurrency for the threaded writers; the whole dataset is one portable file for submission.
Chart.js (vendored)	Charts in dashboard & reports	Beautiful interactive charts with no build step; vendoring the single JS file makes dashboards and generated reports work fully offline.
Paramiko	Real SSH server + client handshakes	The only mature pure-Python SSH library; gives authentic key exchange and auth so captured SSH events are genuine protocol interactions.
requests	Optional public-IP geolocation	Only used when the user opts in; lazy-imported so the core works without it.
stdlib http.server / socket / threading	HTTP decoy, sensors, concurrency	No external daemons; each listener is an independent thread — simple and robust at lab scale.
Docker / docker-compose	Optional isolated deployment	Reproducible environment; decoys and dashboard ship as one image; volumes persist data.
unittest	Automated testing	Part of the standard library — zero extra dependency; 34 tests cover parser, scoring, storage and pipeline.

9. Installation & Setup

9.1 Prerequisites

- Python 3.10 or newer (developed and tested on 3.14).
- pip and an internet connection only for the first-time dependency install.
- Any OS — macOS, Linux or Windows. No administrator/root rights needed with the default ports.

9.2 Install and first run

```
cd honeypot-dashboard
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

python run.py                       # start decoys + dashboard
```

Open **http://127.0.0.1:5555**. The dashboard opens empty (0 events) — this is correct; the project only shows genuinely captured traffic.

9.3 Dependencies

```
flask>=3.0      # dashboard
paramiko>=3.4   # real SSH handshakes
requests>=2.31  # optional geolocation lookup
```

10. Starting the Demonstration (Step-by-Step)

The following is the recommended 5–7 minute flow. Everything shown to the audience is real traffic captured live — nothing is pre-loaded.

Step	Action	Purpose
1	Open a terminal and run: python run.py	Starts all decoys and the dashboard on http://127.0.0.1:5555.
2	Open the dashboard in TWO tabs: Overview and Events	One shows live KPIs/charts, the other the detailed log.
3	Confirm the dashboard shows 0 events	Proves the data is genuine from this point on.
4	In a second terminal run: python scripts/live_demo.py	Launches the narrated, interactive attack demo.
5	Press Enter between scenarios while explaining each attack	Controls pacing; each scenario is a distinct real attack.
6	After the last scenario, go to Security Report → Generate report now	Shows the aggregated findings and recommendations.
7	Reset for the next run: python run.py --mode reset-db	Returns to a clean slate (or use the Clear all events button).

11. Simulating Attacks Locally

11.1 The narrated demo (recommended)

```
python scripts/live_demo.py          # interactive, press Enter between scenes
python scripts/live_demo.py --auto   # non-interactive rehearsal
```

11.2 Individual scenarios

```
python scripts/simulate_attacks.py --all               # every attack type at once
python scripts/simulate_attacks.py --scenario ssh      # SSH brute force
python scripts/simulate_attacks.py --scenario port-scan
python scripts/simulate_attacks.py --scenario stuffing # credential stuffing
python scripts/simulate_attacks.py --scenario web      # SQLi/XSS/traversal
python scripts/simulate_attacks.py --scenario ftp
python scripts/simulate_attacks.py --scenario telnet
```

11.3 Using real external tools (optional)

```
nmap -p 1433,3306,3389,5432,5900,6379,8000,8443,9000,9200,27017 127.0.0.1
curl "http://127.0.0.1:8081/product.php?id=1' OR '1'='1"
hydra -l root -P rockyou.txt 127.0.0.1 -s 2222 ssh -t 4     # if installed
```

Each scenario binds its outbound connections to a distinct REAL local address (the machine's LAN IP plus loopback), so the dashboard shows multiple attacker identities — genuine connections, not fabricated rows.

12. Simulating Attacks From Another Laptop

Attacking from a second machine is the most convincing demonstration because the victim sees the attacker's **real, separate IP address**. Only do this on a network you own or are authorized to monitor.

12.1 Find the honeypot machine's LAN IP

```
# macOS
ipconfig getifaddr en0
# Linux
hostname -I
# Windows
ipconfig | findstr IPv4
```

12.2 Expose the decoys on the LAN (with consent)

By default the decoys bind only to loopback. Binding to all interfaces requires the explicit consent flag, which is the project's ethical gate:

```
# set the decoy bind address in config.json to "0.0.0.0", then:
python run.py --i-own-this-network
```

On macOS the OS may prompt to allow Python to accept incoming connections — click Allow. Ensure any local firewall permits the decoy ports (2222, 8081, 2121, 2323 and the sensor ports).

12.3 Attack from the second laptop

```
# SSH brute force (from laptop 2)
ssh root@<HONEYPOT_LAN_IP> -p 2222
# Web login / credential stuffing
curl "http://<HONEYPOT_LAN_IP>:8081/login" -d "username=admin&password=admin"
# SQL injection
curl "http://<HONEYPOT_LAN_IP>:8081/item?id=1' OR '1'='1"
# Port scan
nmap -p 1433,3306,3389,5432,6379,8000,8443,9200 <HONEYPOT_LAN_IP>
# FTP
ftp <HONEYPOT_LAN_IP> 2121
```

12.4 Verify on the dashboard

- The attacker's LAN IP (e.g. 192.168.1.20) now appears on Overview, Events and Attacker Intel, labelled “Private network”.
- Open Events → filter severity = CRITICAL and click rows to show the captured credentials/payloads.
- Re-running the guided demo with the second laptop as an extra attacker mixes both identities on screen.

---

13. The Live Demo Script (Scenario-by-Scenario)

scripts/live_demo.py narrates each stage and pauses for Enter. What each scenario does, and what to point at on the dashboard:

#	Scenario	Real traffic sent	Dashboard observation
1	Port scan	11 real TCP connects across sensor ports from a real local address	Events → filter service=sensor; “port_scan” appears in the Attack Patterns chart.
2	SSH brute force	8 real SSH handshakes, each with a different username/password	Events → severity HIGH/CRITICAL; open a row to show root/toor, admin/admin, …
3	Credential stuffing	8 real HTTP POSTs to the decoy login form	Credentials page — username/password chart and combination table fill live.
4	Web app attacks	SQLi, XSS, path-traversal and webshell probes (sqlmap signature)	Events → read the raw payload in the detail modal.

#	Scenario	Real traffic sent	Dashboard observation
5	FTP + Telnet	Real USER/PASS conversations on :2121 and :2323	New ftp/telnet login_attempt events with the credentials.
6*	Command capture	Attacker opens an interactive shell and types commands	Events → filter type=command; show exactly what was typed.

* Scenario 6 runs automatically only when services.ssh.accept_any_credentials is enabled; otherwise it prints a one-line explanation and skips. See section 17 for how the fake shell remains safe (nothing is ever executed).

14. Dashboard Tour

Page	Contents
Overview	KPI cards (total events, unique IPs, last-24h, critical, credential attempts), 48-hour attack timeline, severity doughnut, service distribution, top-10 attacker IPs, attack-pattern ranking, live event feed, honeypot listener health. Auto-refreshes every 5 seconds.
Events	Full event log with free-text search plus severity/service/pattern filters, pagination, and a click-to-open detail modal (credentials, payload, user-agent, geolocation). Includes CSV/JSON export and the Clear all events button.
Attacker Intel	Per-IP aggregation: location, ASN, totals, worst severity, first/last seen, one-click jump to that IP's log.
Credentials	Top attempted usernames/passwords bar chart and the username+password combination table.
Security Report	One-click report generation plus CSV/JSON dataset export links.

15. Security Report Generation

The Report page (or POST /api/report/generate) produces a timestamped self-contained HTML report whose Chart.js is embedded inline, so it opens and prints to PDF fully offline. A matching Markdown summary is also written to reports/out/.

Report element	Detail
Executive summary	Total events, unique IPs, critical events, credential attempts, commands captured.
Charts	72-hour timeline, severity, service, attack-pattern, top usernames/passwords.
Top attacker IPs	IP, worst severity, event count, country, last seen.
Credentials	Most attempted usernames and passwords (weak ones are highlighted).
Recommendations	Rule-based defensive advice: fail2ban/lockout for brute force; MFA + rate-limiting for stuffing; firewall hardening + IDS for scans; WAF + input validation for web attacks; segmentation for captured commands.

16. Testing & Evaluation

16.1 Automated tests

```
python -m unittest discover -s tests -v     # 34 tests, all passing
```

Coverage: parser normalisation + Cowrie/auth.log import; severity threshold boundaries; brute-force/stuffing/scan detection; web payload signatures; storage filters, pagination, per-IP aggregates;

end-to-end pipeline escalation; historical replay scoring; the ethical consent gate.

16.2 Manual evaluation cases (TC1–TC8)

Case	Stimulus	Expected result
TC1	SSH brute force (simulator or hydra)	login_attempt events; ≥5th tagged brute_force; severity escalates.
TC2	Port scan (simulator or nmap)	port_probe events; ≥5 distinct ports → port_scan.
TC3	HTTP credential stuffing	POSTs captured; ≥6 distinct usernames → credential_stuffing (CRITICAL).
TC4	Web payload attacks	sql_injection / xss / path_traversal / webshell_probing patterns.
TC5	FTP + Telnet logins	login_attempt events with the exact credentials.
TC6	Interactive shell commands	login_success + command events; nothing executed.
TC7	Report generation	HTML + Markdown files produced with correct KPIs.
TC8	Consent gate	Non-loopback binding refused without --i-own-this-network.

17. Safety, Ethics & Legal

- Deployment is restricted to machines/networks you own or are explicitly authorized to monitor.
- Decoys bind loopback by default; exposing them on other interfaces requires the --i-own-this-network consent flag.
- The simulator refuses targets that are not loopback/private without the same flag.
- The SSH fake shell is pure string matching — there is no subprocess, os.system or eval anywhere in the codebase, so attacker input can never execute.
- Payloads are truncated (4 KB), credentials are stored only as captured from decoy interactions, and nothing is uploaded — data stays in data/ and logs/.
- Legal note (India): unauthorized access is an offence under the Information Technology Act, 2000 (ss. 43, 66). The project's guardrails exist to keep experimentation lawful. See docs/ethics_and_safety.md.

18. Troubleshooting & FAQ

Symptom	Cause / fix
“Address already in use” on a decoy port	Another app uses the port (macOS AirPlay uses 5000/7000; many dev stacks use 8080). Change the port in config.json or via the CLI flags; busy sensors are skipped and shown as stopped on Overview.
Dashboard not reachable	Check the terminal for the actual URL/port printed at startup.
No data appears on the dashboard	Run scripts/live_demo.py or the simulator; check /api/status shows listeners running.
macOS loopback aliases (127.0.0.2…) unavailable	macOS exposes only 127.0.0.1 by default; the demo automatically falls back to the LAN IP. Add aliases with: sudo ifconfig lo0 alias 127.0.0.2 up.
SSH handshake error “No existing session”	The client socket must be connected before being passed to paramiko (fixed in the simulator).
Want attacker countries on the Intel page	Set geo_lookup_enabled: true (needs internet; results are cached).

Symptom	Cause / fix
Reset between demos	python run.py --mode reset-db, or the Clear all events button on the Events page.

19. Limitations & Future Scope

Limitations: low-interaction only (no kernel/OS-level deception); IPv4 focus; rule-based scoring rather than machine learning; single-node deployment; not hardened against hostile public-internet exposure.

Future scope: threat-intelligence enrichment (AbuseIPDB), automatic firewall blocking, ML-based clustering of attacker campaigns, alerting via email/Telegram, and multi-node collection with a central aggregator — the exported CSV dataset is already structured for these extensions.

20. References

OWASP Python Honeypot — https://github.com/OWASP/Python-Honeypot

Cowrie SSH/Telnet Honeypot — https://github.com/cowrie/cowrie

The-HoneyPot — https://github.com/neelspatel999/The-HoneyPot

Flask documentation — https://flask.palletsprojects.com

SQLite documentation — https://www.sqlite.org/docs.html

Chart.js documentation — https://www.chartjs.org/docs/

Paramiko documentation — https://www.paramiko.org

Appendix A — Project File Structure

```
honeypot-dashboard/
├── run.py                      # entry point (all modes)
├── core/
│   ├── config.py               # defaults + overrides
│   ├── database.py             # SQLite storage layer
│   ├── parser.py               # normalisation + Cowrie/auth.log importers
│   ├── threat_scoring.py       # scoring / severity / pattern engine
│   ├── geoip.py                # private-IP labels + opt-in lookup
│   └── collector.py            # pipeline: normalize→score→enrich→store
├── honeypot/
│   ├── base_sensor.py          # generic TCP sensor, FTP + Telnet decoys
│   ├── ssh_honeypot.py         # paramiko SSH decoy (+ optional fake shell)
│   ├── http_honeypot.py        # HTTP decoy with credential capture
│   └── listener_manager.py     # lifecycle + status of all listeners
├── dashboard/
│   ├── app.py                  # Flask factory & pages
│   ├── api.py                  # REST API (JSON)
│   ├── templates/              # Jinja2 pages
│   └── static/                 # CSS, JS, vendored Chart.js
├── reports/generator.py        # self-contained HTML + MD reports
├── scripts/
│   ├── simulate_attacks.py     # controlled attack simulator
│   ├── live_demo.py            # narrated teacher demo
│   ├── seed_demo_data.py       # OPTIONAL synthetic preview
│   └── generate_pdf.py         # builds this PDF
├── tests/                      # 34 unit + integration tests
├── docs/                       # architecture, guides, demo, ethics, viva
├── data/                       # created at runtime (SQLite db)
├── logs/                       # created at runtime (raw JSONL)
└── reports/out/                # generated security reports
```

Appendix B — Command Quick Reference

```
# start / stop
python run.py                         # decoys + dashboard
python run.py --mode dashboard        # dashboard only
python run.py --mode honeypot         # decoys only
python run.py --mode reset-db         # wipe database
python run.py --mode import --file cowrie.json --format cowrie

# dashboard
http://127.0.0.1:5555                 # overview
.../events  .../intel  .../credentials  .../report

# demonstrations
python scripts/live_demo.py                          # narrated (Enter-paced)
python scripts/simulate_attacks.py --all             # every attack at once
python scripts/simulate_attacks.py --scenario ssh    # single scenario

# exports & reporting
/api/export/events.csv   /api/export/events.json     # dataset export
Report page -> "Generate report now"                 # self-contained HTML

# tests
python -m unittest discover -s tests -v

# documentation
python scripts/generate_pdf.py     # rebuild this PDF
```

HoneyScope v1.0.0 · generated by scripts/generate_pdf.py · B.Tech Cyber Security project — Honeypot with Attack Visualization Dashboard.